Report (Assignment 3)

LLMs and KGs

Georgios Boufis

1.Introduction

This notebook implements a complete pipeline for extracting entities (Apartment types & Amenities) from real-world shared-housing descriptions and evaluating the performance of the extractor.
Unlike the mock version from the instructor's template, our implementation uses the **OpenAI API (gpt-4o-mini)** both for:

1. **Task 1 — GPT-based Entity Extractor**
   Extracts apartment types & amenities directly from the text using strict rule-based prompting.
2. **Task 2 — Loading and Using the Evaluation Dataset (dataset.json)**
   We load 12 manually annotated texts that contain ground-truth labels for apartments and amenities.
3. **Task 3 — LLM-as-a-Judge Evaluator**
   A second GPT model evaluates whether the extractor's outputs match the ground truth, labeling each term as **correct** or **incorrect**.
   Again, this uses a real API model, not a mock evaluator.

Finally, we compute **precision** and **recall** for both entity types and interpret their meaning.

2. Task 1 — GPT-Based Entity Extractor

Purpose of this component

We built a class GPTEntityExtractor that sends a carefully designed system/user prompt to GPT so that it performs **verbatim extraction**:

1) Extract only terms that appear **exactly** in the text.
2) No hallucinations.
3) No synonyms.
4) Extract terms even if they appear in negated form (“does not include Wi-Fi”).
5) Extract all apartment-related tokens (e.g., both *apartments* and *two-bedroom apartment*).

How the extractor works

The system prompt defines the two classes:

Apartments Terms :

1) apartment
2) apartments
3) studio
4) shared apartment
5) two-bedroom apartment
6) three-bedroom apartment
7) two-room apartment

Amenity Terms :

1) Wi-Fi
2) parking
3) heating
4) balcony

5) Garden

Example Output Interpretation

Input text:

“The apartments include Wi-Fi and parking. It is a two-bedroom apartment near the metro station.”

Extractor output:

```
{
   "apartments": ["apartments", "two-bedroom apartment"],
   "amenities": ["Wi-Fi", "parking"]
}
```

This is **fully correct**:
“apartments” appears
“two-bedroom apartment” appears
“Wi-Fi” appears
“parking” appears

This shows that the extractor behaves exactly as instructed.

Task 2 - Loading & Understanding the Evaluation Dataset

We load dataset.json, which includes **12 texts**, each with:

- ground_truth_apartments
- ground_truth_amenities

These were manually annotated and serve as the “gold labels” used to evaluate extractor performance.

Example from the dataset

"text": "A cozy studio comes with heating and a balcony.",
"ground_truth_apartments": ["studio"],
"ground_truth_amenities": ["heating", "balcony"]

These ground truths are what the Judge must compare to the extractor’s output.

Task 3 - LLM-as-a-Judge Evaluator

We implemented a second class: GPTJudge

Purpose :

This model **does not extract entities**.
Its only job is to judge each extracted term as:

- **correct** → the term exists in ground truth
- **incorrect** → the term is not in ground truth

Why this is important :

This allows us to simulate a trusted evaluator that avoids:

- hallucinating new entities
- interpreting meaning
- inferring extra information

It strictly performs **set membership checking**.

Example Evaluation Output :

For text:

“A two-bedroom apartment includes heating but does not include Wi-Fi and parking.”

Extractor output :

'apartments': ['two-bedroom apartment'],

'amenities': ['heating', 'Wi-Fi', 'parking']

Ground truth :

'apartments': ['two-bedroom apartment'],

'amenities': ['heating']

Judge output :


```
{
  "apartments": [
    {"term": "two-bedroom apartment", "label": "correct"}
  ],
  "amenities": [
    {"term": "heating", "label": "correct"},
    {"term": "Wi-Fi", "label": "incorrect"},
    {"term": "parking", "label": "incorrect"}
```

]

}

Why “Wi-Fi” and “parking” are incorrect ?

Because although they appear in the text, the **ground truth** says they should NOT be extracted as amenities (due to being negated).
The extractor obeys the rule *“extract even negated terms”*, while the ground truth is based on *positive mentions only*.

Thus, mismatch = incorrect.

Precision & Recall Analysis :

a) Precision :

Precision = TP / (TP + FP)
→ Out of what the extractor extracted, how many were correct?

Low precision means:
Extractor added extra entities that were not in ground truth.

b) Recall :

Recall = TP / (TP + FN)
→ Out of all the ground-truth entities, how many did the extractor find?

Recall tends to be very high (often 1.0) because:
1) The extractor rarely **misses** an entity from the ground truth.
2) But it often **adds extra terms** (FP), which affects precision, not recall.

Detailed Interpretation of the Outputs :

Here is what we observe across the 12 texts:

Apartment Extraction :

Almost all texts show:

- **Precision = 1.0**
- **Recall = 1.0**

Reason :

Apartment terms appear clearly and the extractor catches all of them without adding hallucinations.

Example :
Ground truth :

["apartments", "two-bedroom apartment"]

Extractor :

["apartments", "two-bedroom apartment"]

Perfect match → Precision = 1, Recall = 1

Amenity Extraction :

Amenity extraction is where most errors happen.

Common Pattern :

Text:

“includes heating but does not include Wi-Fi and parking.”

Extractor :

['heating', 'Wi-Fi', 'parking']

Ground truth:

['heating']

Precision:

TP = 1 (“heating”)
FP = 2 (“Wi-Fi”, “parking”)
→ Precision = 1 / 3 = **0.33**

Recall:

TP = 1
FN = 0
→ Recall = **1.0**

Why recall = 1 ?

Because recall only checks whether the extractor successfully found the POSITIVE terms in ground truth.

Ground truth amenities list:

- heating

Extractor found:

- heating
  → So recall is perfect.

Why precision is low ?

Because the extractor *correctly* obeys the rule:

"Extract amenities even if they appear in negated form."

But ground truth does NOT include negated amenities.

Thus, extractor produces FP → lowers precision.

GPT Extractor Performance :

Extractor is **very reliable** for apartment terms.

Extractor is **over-inclusive** with amenities due to rule that extracts negated

mentions.

This behavior is expected, not a failure — it follows the designed guidelines.

GPT Judge Performance :

Judge perfectly labels correct/incorrect based on ground truth.
It does not hallucinate or reinterpret negations.
It strictly evaluates based on membership comparison.

Precision and Recall Summary :

Apartment Precision = 1.0, **Recall = 1.0**
→ Extractor always gets apartments right.
Amenity Precision varies (0.33, 0.5, 1.0)
→ Because extractor includes negated amenities that ground truth excludes.
Amenity Recall = 1.0
→ The extractor always finds all positively mentioned amenities.

Interpreting the Results :

Below is a human-readable, clear explanation of what happens in each case.

Text 1

Extracted:

- apartments: ['apartments', 'two-bedroom apartment']
- amenities: ['Wi-Fi', 'parking']

Ground truth: identical.

Judge output: all "correct".

The extractor matched the ground truth perfectly.

Text 2

Same situation: perfect match.

Both entities and labels are correct.

Text 3

Again, perfect match.

Extractor and ground truth fully align.

Text 4

Extracted amenities: ['heating', 'balcony', 'Wi-Fi']
Ground truth: ['heating', 'balcony']

The text contains:
"... a small balcony **but no Wi-Fi**."

Because the extractor is instructed to **extract ALL verbatim terms**, even in negations, it correctly extracted "Wi-Fi".
However, the ground truth includes only *positive* amenities.

So:

Wi-Fi" → **incorrect**
The rest → correct
This is expected behaviour.

Text 5

All labels "correct".

Extractor matches ground truth.

Text 6

All labels "correct".

No disagreement.

Text 7

Extracted amenities:
['Wi-Fi', 'heating', 'parking']

Ground truth amenities:
['Wi-Fi', 'heating']

The text mentions *private parking*, but the ground truth does **not** include "parking".

Therefore:

- "Wi-Fi" → correct
- "heating" → correct
- "parking" → **incorrect**

Judge is correct — it checks ground truth only, not the text.

Text 8

Extractor found:
['Wi-Fi', 'heating', 'balcony']

Ground truth only contains:
['balcony']

So:

- "balcony" → correct
- "Wi-Fi" → incorrect
- "heating" → incorrect

Again consistent with ground truth.

Text 9 - 12

The same pattern continues:

- Terms present in both lists → correct
- Terms that the extractor found but ground truth did not include → incorrect
- No hallucinations, no extra invented terms.

Overall Observations :

Extractor Behaviour :

- It extracts **all** relevant terms it sees, exactly as the prompt instructs.
- It never hallucinated.
- It always preserved the exact form (hyphens, capitalization).
- This leads to **very high recall**, because it never misses a ground truth term.

Judge Behaviour :

- The judge is perfectly consistent:
  it only checks membership in the ground truth and returns correct/incorrect.
- It does not extract, modify, or infer anything.
- It never adds or removes entities.

Disagreements :

Most incorrect labels occur because **ground truth only includes positive amenities**, while the extractor includes *all mentions*, including negated ones.

This is expected and shows that:

- Extractor = text-oriented
- Ground truth = semantics-oriented
- Judge = strict comparator

Overall System Accuracy :

The extractor performs extremely well in recall, and reasonably well in precision, given the differences in logic between extraction rules and ground truth definition.